

NoSQL — Unit 1 Notes

Module 1–4: Foundations of NoSQL, Aggregate Data Models & Exam Preparation

What is SQL?

SQL (Structured Query Language) databases are the relational databases that store data in tables with fixed schemas, use SQL as the query language, and follow ACID properties for transactions.

Key characteristics of SQL/Relational databases:

Feature	Explanation
Structured data	Data must fit into predefined tables with rows and columns
Fixed schema	You must define the structure (columns, data types) before inserting data
Normalization	Data is split into multiple tables to eliminate redundancy
JOINS	Related data is combined at query time using JOIN operations
ACID transactions	Atomicity, Consistency, Isolation, Durability — guaranteed
Vertical scaling	To handle more load, you buy a bigger, more powerful server
Strong consistency	Every read receives the most recent write

Examples: MySQL, PostgreSQL, Oracle, SQL Server, SQLite.

The problem with SQL databases at scale:

1. **Vertical scaling has limits** — you can only buy so big a server.
2. **Fixed schema is rigid** — changing schema on a live database with millions of rows is painful.
3. **JOINS are expensive** — as data grows across many tables, JOIN operations become slow.
4. **ACID is expensive in distributed systems** — maintaining strong consistency across multiple machines requires complex coordination.
5. **Impedance mismatch** — object-oriented code doesn't map cleanly to tables.

What is NoSQL?

NoSQL databases are non-relational, distributed database systems designed to handle large volumes of structured, semi-structured, and unstructured data. They prioritize **availability**, **scalability**, and **performance** over strict **consistency** and **ACID guarantees**.

When did NoSQL emerge?

- The term gained popularity around **2009**.
- It emerged as a response to the needs of **web-scale companies** (Google, Amazon, Facebook) who found relational databases inadequate for their massive, rapidly growing, semi-structured data.

What is Unique About NoSQL? (How It Differs from SQL)

This is the **most important comparison** for the exam. Here is a detailed point-by-point comparison:

Aspect	SQL (Relational)	NoSQL
Data Model	Tables with rows and columns (flat, normalized)	Key-value, document, column-family, graph (aggregate-oriented or graph)
Schema	Fixed, predefined schema	Flexible or schema-less
Query Language	SQL (standardized)	Varies by database (some have APIs, some have SQL-like languages)
Scaling	Vertical (bigger server) — limited and expensive	Horizontal (more servers) — virtually unlimited and cheap
Consistency	Strong consistency (ACID)	Eventual consistency (BASE) — trade-off for availability
Transactions	Full ACID support across multiple tables	Limited or no multi-record transactions
JOINS	Native support	Typically not supported — data is denormalized

Normalization	Data is normalized to reduce redundancy	Data is denormalized for query performance
Data Volume	Handles moderate data well	Designed for massive data (terabytes to petabytes)
Data Variety	Structured data only	Structured, semi-structured, unstructured
Distribution	Typically single-node	Designed for distributed clusters
Use Case	Banking, ERP, applications needing strict consistency	Real-time web apps, big data, IoT, social networks
Examples	MySQL, Oracle, PostgreSQL	MongoDB, Redis, Cassandra, Neo4j

The unique aspects of NoSQL summarized:

1. **Schema flexibility** — You can store different structures in the same collection/table.
2. **Horizontal scalability** — Add more commodity servers instead of upgrading one big server.
3. **Aggregate-oriented data** — Data that is accessed together is stored together.
4. **BASE instead of ACID** — Basically Available, Soft state, Eventually consistent.
5. **Designed for clusters** — Distribution and replication are built into the architecture.
6. **Polyglot persistence** — Different databases for different needs within the same system.

ACID vs BASE

ACID (SQL databases):

Property	Meaning
Atomicity	A transaction is all-or-nothing
Consistency	Data always moves from one valid state to another
Isolation	Concurrent transactions don't interfere with each other
Durability	Once committed, data survives system failures

BASE (NoSQL databases):

Property	Meaning
Basically Available	The system guarantees availability (responds to every request, even if it might fail)
Soft state	The state of the system may change over time, even without input, due to eventual consistency
Eventually consistent	Given enough time, all replicas will converge to the same value
ACID	Correctness and consistency
BASE	Availability and Scalability

The Value of Relational Databases

Even though NoSQL exists, relational databases remain valuable because:

1. **Mature and proven** — Decades of optimization and reliability.
2. **Standardized** — SQL is universal; skills transfer easily.
3. **Strong consistency** — Essential for financial, medical, and legal applications.
4. **Complex queries** — JOINS, aggregations, subqueries are powerful.
5. **Tooling ecosystem** — ORMs, reporting tools, admin interfaces.
6. **Data integrity** — Foreign keys, constraints, and ACID prevent corruption.

Impedance Mismatch

Impedance mismatch is the set of difficulties that arise when the object-oriented model of an application does not align with the relational model of the database.

Where it occurs:

Application (OOP)	Database (Relational)
-------------------	-----------------------

Objects	Tables
Classes	Schemas
Inheritance	No direct equivalent
References (pointers)	Foreign keys
Collections (lists, sets)	Separate tables with foreign keys
Encapsulation	Public columns

Example

Imagine a Customer object in Java:

```
class Customer {
    String name;
    List<Address> addresses;
    List<Order> orders;
}
```

In a relational database, this becomes:

- `customers` table
- `addresses` table (with `customer_id` foreign key)
- `orders` table (with `customer_id` foreign key)

To reconstruct the object, you need multiple JOINS. This is the impedance mismatch.

Application Databases vs Integration Databases

Integration Database

A database that is shared by multiple applications and serves as a central point of data integration.

- Data is a shared resource.
- Schema must accommodate all applications.
- Changes affect all consumers.
- Example: A corporate ERP database used.

Characteristics

- Strong schema enforcement.
- ACID transactions critical.
- Typically relational.
- Changes are carefully managed.

Application Database

A database is owned and accessed by a single application.

- The application controls the schema.
- The database is behind the application's API.
- Changes affect only one application.
- Example: A session store for a web application.

Characteristics

- Schema can be optimized for the application's needs.
- No need to coordinate with other applications.
- NoSQL databases are often well-suited here.

NoSQL databases are more commonly used as **application databases** because they can be tailored to specific access patterns without worrying about other consumers. Integration databases still tend to be relational.

Getting at Persistent Data

Applications need to store data so that it survives after the program stops running — this is called **persistent data**. Understanding how applications have historically interacted with persistent data helps explain why relational databases became dominant, and why NoSQL later emerged as an alternative.

The core idea: separation of data storage from application logic.

From early in the history of computing, it became standard practice to keep the data storage system **separate** from the application that uses the data. Rather than each application inventing its own way to save and organize files, applications talk to a dedicated **database server** through a well-defined interface (such as SQL).

Why this separation matters:

Reason	Explanation
Data outlives applications	Data is often needed long after the program that created it has been rewritten or replaced
Shared access	Multiple applications and multiple users often need to read and write the same data at the same time
Data integrity	A dedicated database server can enforce rules (schema, constraints) so that all applications see consistent, valid data
Simplifies applications	Application developers do not need to write low-level file-handling or storage code themselves
Random access	Databases are optimized to quickly locate and update a specific piece of data, not just read files start to finish

Historical progression:

- Early file-based systems** — Applications read and wrote their own custom file formats directly; there was no shared, general-purpose database server.
- Rise of the database server** — A separate server process became responsible for storing data, handling concurrent access, and enforcing structure — freeing applications from needing to manage storage themselves.
- Relational databases standardize this model** — By presenting data as tables and offering SQL as a common interface, relational databases became the default way for applications everywhere to persist data.

Exam Point: The separation of persistent data storage from application logic is a foundational idea that both relational and NoSQL databases build on — NoSQL does not abandon this separation, it changes what happens on the storage side of that boundary.

Concurrency

Concurrency refers to how a database manages the situation where more than one process (or user) tries to access or modify the same data at the same time. Without proper concurrency control, simultaneous access can corrupt data or produce inconsistent results.

Why concurrency matters:

Real-world applications are almost never used by just one person at a time. Databases must ensure that concurrent reads and writes do not interfere with each other in ways that leave the data in an incorrect or inconsistent state.

Two broad approaches to concurrency control:

Approach	How It Works	Trade-off
Pessimistic Concurrency	Assumes conflicts are likely, so it locks data before it is accessed, preventing others from reading or writing it until the lock is released	Safer against conflicts, but reduces performance since other processes must wait for locks to be released
Optimistic Concurrency	Assumes conflicts are rare, so it lets multiple processes proceed without locking, then checks for conflicts before finally committing a change	Better performance and higher throughput, but must handle the case where a conflict is detected (e.g., by retrying or rejecting the update)

Pessimistic concurrency in more detail:

- A process must acquire a **lock** on a piece of data before reading or writing it.
- Other processes wanting to access the same data must wait until the lock is released.
- Guarantees correctness but can create bottlenecks, especially in distributed systems where locking across machines is slow and fragile.

Optimistic concurrency in more detail:

- Processes read and work with data freely, without taking a lock first.
- Before committing a change, the system checks whether the data has been modified by someone else since it was read (often using a version number or timestamp).

- If a conflict is detected, the update is rejected or retried; if not, the change is committed.

Why this matters for NoSQL:

Because NoSQL databases are designed to run across **distributed clusters**, pessimistic locking across many nodes is often impractical — it is slow and creates coordination overhead. As a result, many NoSQL databases favor **optimistic concurrency** or accept **eventual consistency** (BASE) rather than strictly locking data, trading strict correctness guarantees for better availability and performance.

Exam Point: Pessimistic concurrency favors correctness through locking; optimistic concurrency favors performance by checking for conflicts only at commit time. NoSQL's preference for horizontal scaling pushes it toward optimistic, loosely-coordinated approaches to concurrency.

A (Mostly) Standard Model

For roughly three decades before the rise of NoSQL, the **relational data model** was the undisputed, standardized way that the software industry stored data. Understanding how it achieved this dominance explains why moving to NoSQL was such a significant shift.

How the relational model became standard:

1. **Strong theoretical foundation** — The relational model was based on well-defined mathematical concepts (set theory and predicate logic), giving it academic rigor and credibility.
2. **Commercial backing** — Major vendors (IBM, Oracle, Microsoft, and others) invested heavily in building robust, well-supported relational database products.
3. **Standardized query language** — SQL became a common, portable language that worked (with minor variations) across different vendors' databases, so skills and tools transferred easily.
4. **Proven reliability** — Relational databases offered strong ACID guarantees, which made them trustworthy for critical business data such as banking, inventory, and payroll.
5. **Rich tooling ecosystem** — Reporting tools, admin interfaces, ORMs, and business intelligence software were all built around the relational model, reinforcing its position.

The result: a near-universal default.

Factor	Effect on Standardization
Mathematical foundation	Gave relational databases theoretical credibility and consistent behavior
Vendor competition and investment	Produced mature, well-tested, feature-rich products
SQL as a common language	Made skills and knowledge portable across organizations and products
ACID guarantees	Made relational databases the trusted choice for critical business data
Decades without a serious alternative	Relational became the default assumption — most developers only ever learned this one model

Why this dominance eventually faced challenges:

For most of this period, relational databases worked well because data volumes and traffic could be handled by a single, powerful server (vertical scaling). It was only when **web-scale companies** (Google, Amazon, Facebook, and others) began dealing with massive, rapidly growing, and semi-structured data that the limits of the standard relational model — rigid schemas, expensive JOINS, and the difficulty of scaling horizontally — became serious obstacles. This is what set the stage for the emergence of NoSQL.

Exam Point: The relational model didn't just become popular — it became the **default, standardized** way to store data industry-wide, backed by theory, vendors, and tooling. NoSQL arose specifically because this "one size fits all" standard could not meet the demands of web-scale distributed systems.

Attack of Clusters and the Emergence of NoSQL

The Cluster Problem:

As data volumes grew, companies needed to distribute data across **clusters** of machines. Relational databases were not designed for this:

1. **ACID across nodes is expensive** — Two-phase commit (2PC) is slow and fragile.
2. **JOINS across nodes are impractical** — Data must be co-located for efficient JOINS.
3. **Vertical scaling hits a ceiling** — You can't buy an infinitely powerful server.
4. **Schema rigidity slows change** — Rapidly evolving web applications need flexibility.

The Emergence of NoSQL:

Problem	NoSQL Solution
Vertical scaling limits	Horizontal scaling across commodity servers
Expensive ACID	BASE (eventual consistency)
Rigid schemas	Schema-less or flexible schemas
JOIN complexity	Denormalization (aggregates)
Single-node bottleneck	Built-in sharding and replication
Impedance mismatch	Document/graph models

Key insight: NoSQL databases were designed from the ground up for **distribution, scale**, and **flexibility** — not as an afterthought.

MODULE 2: AGGREGATE DATA MODELS

What is an Aggregate?

An aggregate is a **collection of related objects that we wish to treat as a unit**. It is a cluster of data that is typically accessed and manipulated together.

Why aggregates matter:

In relational databases, data is **normalized** — split into many tables to eliminate redundancy. But this means retrieving a complete business object (like an Order with its Items) requires JOINS across multiple tables.

In NoSQL, data is **denormalized** — stored together as an aggregate. An Order document contains its Items directly. No JOIN needed.

Example of Relations and Aggregates

Relational Model (Normalized)

customers table:

customer_id	name	email
1	Alice	alice@email.com

addresses table:

address_id	customer_id	city	zip
10	1	Mumbai	400001
11	1	Pune	411001

orders table:

order_id	customer_id	date	total
100	1	2025-01-15	5000

order_items table:

item_id	order_id	product	qty	price
1	100	Laptop	1	4500
2	100	Mouse	1	500

To get a complete customer profile with addresses and orders, you need **JOINS across 4 tables**.

Aggregate Model (Denormalized - document)

<pre>{ "customer_id": 1, "name": "Alice", "email": "alice@email.com", "addresses": [{ "city": "Mumbai", "zip": "400001" }, { "city": "Pune", "zip": "411001" }] }</pre>

```
"orders": [
  {
    "order_id": 100,
    "date": "2025-01-15",
    "total": 5000,
    "items": [
      { "product": "Laptop", "qty": 1, "price": 4500 },
      { "product": "Mouse", "qty": 1, "price": 500 }
    ]
  }
]
```

One read retrieves everything. No JOINS. This is the power of aggregates. The aggregate model trades **redundancy** (data may be duplicated) for **query performance** (no JOINS needed).

Consequences of Aggregate Orientation

Advantages

- 1. **Simpler queries** — No JOINS needed; one read gets the whole aggregate.
- 2. **Better performance** — Data is co-located; no cross-node JOINS.
- 3. **Natural sharding** — The aggregate is the sharding unit; data accessed together is stored together.
- 4. **Easier application development** — Objects map directly to aggregates.
- 5. **Schema flexibility** — Each aggregate can have a different structure.

Disadvantages

- 1. **Limited query flexibility** — You can only query efficiently by the aggregate's key or indexed fields.
- 2. **Data redundancy** — The same data may be duplicated across aggregates.
- 3. **No cross-aggregate transactions** — Atomic updates across multiple aggregates are not supported (or are very limited).
- 4. **Difficult ad-hoc queries** — Questions that cut across aggregates (e.g., "total sales by product") require MapReduce or materialized views.
- 5. **Update anomalies** — If duplicated data is updated in one place but not another, inconsistency arises.

Key-Value Data Model

A key-value store is the simplest NoSQL data model. Data is stored as a collection of **key-value pairs**. The key is a unique identifier; the value is an arbitrary data item (string, number, JSON, binary, etc.).

How it works

Key	Value
user:1001	{ "name": "Alice", "age": 30 }
session:abc123	{ "user_id": 1001, "expires": "..."}
cart:1001	["item1", "item2", "item3"]

The entire **value** is the aggregate. The database does not inspect or understand the internal structure of the value.

Features

Feature	Description
Simplicity	Just put(key, value) and get(key)
Performance	Very fast — O(1) lookup by key
Scalability	Easily distributed by hashing the key
Schema-less	Values can be anything
No query language	Only key-based access (and maybe some range queries)
Consistency	Typically eventual consistency, but some (like Redis) offer strong consistency

Transactions: Usually limited to single-key operations.

Query Features: Typically only get, put, delete by key. Some support range queries on keys.

Scaling: Horizontal scaling is trivial — hash the key to determine which node stores it.

Suitable Use Cases:

- **Session information** — Store session data with session ID as key.
- **User profiles** — Store user preferences.
- **Shopping cart data** — Store cart items with user ID as key.
- **Caching** — Fast access to frequently used data.

When NOT to Use:

- **Relationships among data** — Key-value stores can't traverse relationships.
- **Multi-operation transactions** — No ACID across multiple keys.
- **Query by data** — You can only query by key, not by value contents.
- **Operations by sets** — No set-based operations like UNION, INTERSECT.

Examples: Redis, Riak, Amazon DynamoDB (key-value mode), Memcached

Document Data Model

A document database stores data as **documents** — self-contained, self-describing units of data typically formatted as JSON, BSON, or XML. Each document contains **fields** (key-value pairs) and can have **nested structures** (sub-documents, arrays).

How it works

```
{
  "_id": "user:1001",
  "name": "Alice",
  "age": 30,
  "addresses": [
    { "city": "Mumbai", "zip": "400001" },
    { "city": "Pune", "zip": "411001" }
  ],
  "orders": [
    { "order_id": 100, "total": 5000 }
  ]
}
```

The **document** is the aggregate. Unlike key-value stores, the database **understands** the internal structure of the document — it can index fields, query by field values, and update individual fields.

Features

Feature	Description
Schema-less	Different documents can have different fields
Queryable	Can query by field values, not just key
Indexing	Can create indexes on any field
Nested data	Supports sub-documents and arrays
Aggregation	Supports aggregation pipelines (similar to SQL GROUP BY)

Consistency: Typically eventual consistency, but configurable (MongoDB offers strong consistency on single documents).

Transactions: Multi-document transactions are supported in newer versions (MongoDB 4.0+), but limited compared to SQL.

Query Features: Rich query language — find by field, range queries, regex, aggregation pipelines.

Scaling: Horizontal scaling via sharding. Shard key determines which node stores which documents.

Suitable Use Cases

- **Event logging** — Each event is a document.
- **Content management systems** — Articles, pages, media metadata.
- **Blogging platforms** — Posts, comments, tags.
- **Web analytics / real-time analytics** — User activity events.
- **E-commerce applications** — Product catalogs, user profiles, orders.

When NOT to Use

- **Complex transactions spanning different operations** — Multi-document ACID is limited.

- **Queries against varying aggregate structures** — If documents have wildly different structures, querying becomes difficult.
- Examples:** MongoDB, CouchDB, Couchbase, Amazon DocumentDB.

Column-Family Stores

A column-family store (also called a **wide-column store**) stores data in a **two-level aggregate structure**. The first level is the **row key**, which identifies a row. Each row contains one or more **column families**. Each column family contains a set of **columns** (key-value pairs).

```
Row Key: user:1001
  Column Family: profile
    name: "Alice"
    age: "30"
    email: "alice@email.com"
  Column Family: activity
    last_login: "2025-01-15"
    login_count: "150"
  Column Family: preferences
    theme: "dark"
    language: "en"
```

Key characteristics:

1. **Two-level aggregate:** Row → Column Families → Columns.
2. **Column families are defined at schema creation** — but columns within them are flexible.
3. **Rows can have different columns** — no fixed schema for columns.
4. **Columns are sorted by name** — efficient range scans.
5. **Designed for high write throughput** — writes are appended to disk (log-structured merge trees).

Features

Feature	Description
Flexible columns	Each row can have different columns
High write performance	Optimized for write-heavy workloads
Compression	Column families can be compressed together
Sorted columns	Efficient range queries on column names
Consistency	Typically eventual consistency (Cassandra) or strong consistency (HBase)

Transactions: Limited to single-row operations.

Query Features: Query by row key, range scans on row keys, column family access.

Scaling: Horizontal scaling via sharding on row key.

Suitable Use Cases:

- **Time-series data** — Row key = device ID + timestamp.
- **IoT data** — Sensor readings.
- **Recommendation engines** — User-item interactions.
- **Large-scale write-heavy applications** — Logging, messaging.

When NOT to Use:

- **Complex queries** — No JOINS, limited filtering.
- **Small-scale applications** — Overkill for small data.
- **Read-heavy with complex access patterns** — Not optimized for ad-hoc queries.

Examples: Apache Cassandra, HBase, Google Bigtable, ScyllaDB.

MODULE 3: MORE DETAILS ON DATA MODELS

3.1 Relationships

Definition: A relationship is an association between two or more data entities.

How relationships are handled in different models:

Model	How Relationships Are Handled
Relational	Foreign keys + JOINS
Key-Value	Not supported — must be managed in application code
Document	Embedded documents (denormalized) or references (manual JOINS)
Column-Family	Not directly supported — denormalized
Graph	Native — edges are first-class citizens

Exam Point: The way a database handles relationships is one of the most important differentiators between models. Graph databases are the only ones that treat relationships as first-class citizens.

3.2 Graph Databases

Definition: A graph database stores data as **nodes** (entities) and **edges** (relationships). Both nodes and edges can have **properties** (key-value pairs). Relationships are **first-class citizens** — they are stored natively and can be queried directly.

Structure:

```
Node: Alice (Person)
  - name: "Alice"
  - age: 30
Node: Bob (Person)
  - name: "Bob"
  - age: 28
Edge: Alice → Bob (FRIEND)
  - since: 2020
Edge: Alice → Mumbai (LIVES_IN)
  - since: 2015
```

Aggregate characteristic: Graph databases do **NOT** use aggregate orientation. Data is stored as independent nodes and edges. There is no "aggregate" unit.

Features

Feature	Description
Native relationships	Edges are stored, not computed via JOINS
Fast traversal	Following relationships is O(1) per hop
Flexible schema	Nodes and edges can have different properties
Index-free adjacency	Each node directly references its neighbors

Consistency: Typically ACID-compliant (Neo4j) — graph databases often prioritize consistency.

Transactions: Full ACID support in many graph databases.

Query Features: Graph traversal queries (Cypher, Gremlin, SPARQL).

Scaling: Vertical scaling is common; horizontal scaling is challenging for graph databases.

Suitable Use Cases:

- **Connected data** — Social networks, friend-of-a-friend queries.
- **Routing** — Shortest path, GPS navigation.
- **Dispatch** — Logistics, delivery routing.
- **Location-based services** — "Find nearby restaurants."
- **Recommendation engines** — "People who bought X also bought Y."

When NOT to Use:

- **Simple key-value lookups** — Overkill.
- **Massive write-heavy workloads** — Graph databases are optimized for traversal, not write throughput.
- **Data with no relationships** — If your data is flat, a graph database adds no value.

Examples: Neo4j, Amazon Neptune, ArangoDB, JanusGraph.

Exam Point: Graph databases excel when **relationships are the primary focus** of your queries. They are the opposite of aggregate-oriented databases.

3.3 Schemaless Databases

Definition: A schemaless database (also called **schema-less** or **schema-on-read**) does **not require** a predefined schema. The structure of the data is determined by the application at write time, and interpreted at read time.

Important clarification: Schemaless does **NOT** mean there is no schema. It means the schema is **not enforced by the database**. The application code is responsible for maintaining data consistency.

How it works:

In a schema-less document database, you can insert these two documents into the same collection:

```
{ "name": "Alice", "age": 30, "email": "alice@email.com" }
{ "name": "Bob", "phone": "123-456-7890", "address": "Mumbai" }
```

The first document has `age` and `email`; the second has `phone` and `address`. The database accepts both without complaint.

Advantages:

- 1. **Flexibility** — Adapt to changing requirements without migrations.
- 2. **Speed of development** — No need to design schema upfront.
- 3. **Heterogeneous data** — Store different structures in the same collection.
- 4. **Data integration** — Easily combine data from different sources.

Disadvantages:

- 1. **Application complexity** — The application must handle all schema validation.
- 2. **Data quality risks** — Without database enforcement, bad data can creep in.
- 3. **Query difficulty** — Queries against varying structures are harder to write.
- 4. **Documentation burden** — The "schema" exists only in code and documentation.

Exam Point: Schemaless is a double-edged sword — it gives flexibility but shifts responsibility to the application.

3.4 Materialized Views

Definition: A materialized view is a **precomputed, stored query result** that organizes data in a format optimized for specific query patterns. Unlike a regular view (which is computed on-the-fly), a materialized view is **physically stored** and can be indexed.

Why are they needed in NoSQL?

In aggregate-oriented databases, data is stored in a format optimized for **writes** (aggregates). But queries often need a different format. For example:

- **Stored format:** Orders as aggregates (each order contains all its items).
- **Query need:** "Total sales by product" — requires scanning all orders and summing by product.

A materialized view for this query would store:

Product	Total Sales
Laptop	450,000
Mouse	50,000
Keyboard	75,000

How materialized views work in NoSQL:

- 1. **Generated by MapReduce** — MapReduce jobs scan the base data and produce the view.
- 2. **Read-only** — Applications read from the view but write to the base data.
- 3. **Eventually consistent** — The view is updated periodically or incrementally.
- 4. **Disposable** — Can be deleted and rebuilt from base data at any time.
- 5. **Independent storage** — Can have its own partition key, throughput, and configuration.

Example:

Base data (orders collection):

```
{ "order_id": 100, "items": [{"product": "Laptop", "qty": 1}, {"product": "Mouse", "qty": 1}] }
```

```
{ "order_id": 101, "items": [{ "product": "Laptop", "qty": 2}] }
```

Materialized view (sales_by_product collection):

```
{ "_id": "Laptop", "total_qty": 3 }  
{ "_id": "Mouse", "total_qty": 1 }
```

Exam Point: Materialized views are essential in NoSQL because aggregate-oriented storage is not optimized for all query patterns. They are the NoSQL equivalent of creating a custom index for a specific query.

3.5 Modeling for Data Access

Core Principle: In NoSQL, you model data based on **how it will be accessed**, not on its natural structure. This is the opposite of relational modeling, where you normalize data first and then figure out queries.

Relational Modeling (Data-Centric):

1. Identify entities and relationships.
2. Normalize to eliminate redundancy.
3. Create tables with foreign keys.
4. Write queries using JOINS.

NoSQL Modeling (Query-Centric):

1. Identify **access patterns** (queries the application will run).
2. Design aggregates that make those queries efficient.
3. Denormalize (duplicate) data as needed.
4. Create materialized views for cross-aggregate queries.

Example: An e-commerce application.

Access patterns:

- Get customer by ID → Store customer as a document.
- Get all orders for a customer → Embed orders in customer document (or store orders with customer_id as shard key).
- Get total sales by product → Create a materialized view.

The key insight: The same data may be stored in multiple formats (as different aggregates or views) to satisfy different access patterns. This is called **denormalization** or **polyglot persistence within a database**.

Exam Point: NoSQL modeling is **query-first**. You start with "what questions will I ask?" not "what data do I have?"

MODULE 4: UNIT 1 SUMMARY & EXAM PREPARATION

4.1 Key Definitions to Memorize

Term	Definition
NoSQL	Non-relational, distributed database systems designed for large-scale, flexible, high-performance data storage
Aggregate	A collection of related objects treated as a unit
Impedance Mismatch	The mismatch between object-oriented models and relational tables
Integration Database	A database shared by multiple applications
Application Database	A database owned by a single application
ACID	Atomicity, Consistency, Isolation, Durability
BASE	Basically Available, Soft state, Eventually consistent
CAP Theorem	In a distributed system, you can guarantee at most 2 of: Consistency, Availability, Partition Tolerance
Materialized View	A precomputed, stored query result
Schemaless	No database-enforced schema
Polyglot Persistence	Using different databases for different needs in the same system

4.2 Comparison Tables to Memorize

SQL vs. NoSQL:

SQL	NoSQL
Tables, rows, columns	Key-value, document, column-family, graph
Fixed schema	Flexible schema
Vertical scaling	Horizontal scaling
ACID	BASE
JOINS	Denormalization
Strong consistency	Eventual consistency
Single-node	Distributed

Aggregate Models:

Model	Aggregate	Query By	Best For
Key-Value	Value	Key	Caching, sessions
Document	Document	Fields, key	CMS, e-commerce
Column-Family	Row	Row key	Time-series, IoT
Graph	None	Relationships	Social, routing